



이번 버전은 훨씬 좋아졌습니다. 실제 프레임워크에서 사용할 수 있는 수준으로 다듬어지고 있습니다. 제가 회사 코드 리뷰를 한다는 관점에서 검토해 보겠습니다.

전체 평가

완성도 : 98/100

패키지 구조와 필드 구성 모두 자연스럽습니다.

특히 추가하신 항목들은 매우 좋습니다.

Java



```
private int jdbcType;

private String dbType;

private String columnComments;

private int columnSize;
```

이 네 가지는 앞으로 `ResultSetMetaData` 와 `DatabaseMetaData` 를 사용할 때 매우 유용합니다.

수정하면 좋을 부분

① 오타

</> Java

//데이터베이스 데이터 유형



</> Java

//데이터베이스 데이터 유형



② `columnComments` → `columnComment` 추천

현재

</> Java

```
private String columnComments;
```



가능하지만,

컬럼 하나에 대한 설명이므로 보통은 단수형을 사용합니다.

예를 들면

</> Java

```
private String columnComment;
```



getter/setter

</> Java

```
getColumnComment ()
```

```
setColumnComment ()
```



Oracle의 `COMMENT ON COLUMN`도 하나의 컬럼에 하나의 `Comment`를 의미합니다.

③ displayWidth 기본값

현재

</> Java

```
private int displayWidth = 100;
```



좋습니다.

다만 DataTypeMapper에서

VARCHAR2 (30)

↓

displayWidth = 120



처럼 자동 계산하도록 만들 수도 있습니다.

현재는 그대로 두셔도 됩니다.

④ boolean 기본값

현재

</> Java

```
private boolean editable;  
  
private boolean visible;  
  
private boolean excelExport;  
  
private boolean searchable;
```



Java 기본값은 false입니다.

하지만 업무에서는 대부분

Editable = true

Visible = true

Excel = true



```
Searchable = true
```

가 기본입니다.

그래서 저는

<> Java



```
private boolean editable = true;

private boolean visible = true;

private boolean excelExport = true;

private boolean searchable = true;
```

를 추천드립니다.

그러면 화면에서 별도로 설정하지 않아도 대부분 원하는 동작을 합니다.

⑤ JavaDoc

조금만 보완하면 더 좋습니다.

예를 들어

<> Java



```
//jdbc 데이터 유형
private int jdbcType;
```

↓

<> Java



```
/**
 * JDBC 데이터 유형 (java.sql.Types)
 */
private int jdbcType;
```

또

<> Java



```
//데이터베이스 데이터 유형
private String dbType;
```



Java



```
/**
 * Oracle 데이터 유형
 * (VARCHAR2, NUMBER, DATE ...)
 */
private String dbType;
```

이렇게만 해도 유지보수성이 크게 좋아집니다.

설계적으로 아주 마음에 드는 부분

제가 가장 마음에 든 부분은

jdbcType



javaType



nexacroType

이 3단계입니다.

이 구조는 앞으로

Oracle



ResultSetMetaData



NBDFColumn



DatasetBuilder



Nexacro Dataset

으로 자연스럽게 이어집니다.

앞으로 Mapper에서는 이렇게 사용됩니다.

예를 들면

Java



```
column.setJdbcType(meta.getColumnType(i));  
  
column.setDbType(meta.getColumnTypeName(i));  
  
column.setColumnSize(meta.getPrecision(i));  
  
column.setScale(meta.getScale(i));  
  
column.setNullable(  
    meta.isNullable(i) == ResultSetMetaData.columnNullable  
);
```

이렇게 채워 넣게 됩니다.

한 가지 추가 제안

현재

Java



```
private String comboId;
```

까지 있는데

향후 Grid 자동생성을 생각하면

Java



```
private String defaultValue;
```

도 하나 있으면 좋습니다.

예

Y/N



↓

기본값 = N

또는

등록일

↓

SYSDATE



처럼 활용할 수 있습니다.

하지만 이건 나중에 추가해도 충분합니다.

다음 단계 추천

이제 `NBDFColumn` 은 거의 완성되었으므로, 다음 클래스는 ****NBDFDataTypeMapper****가 가장 적절합니다.

이 클래스가 다음과 같은 역할을 담당합니다.

Oracle ResultSetMetaData

|



NBDFDataTypeMapper

|

├ jdbcType 설정

├ dbType 설정

├ javaType 설정

├ nexacroType 설정

├ columnSize 설정

├ scale 설정

└ nullable 설정



이 클래스가 완성되면 `NBDFColumn` 의 값들이 자동으로 채워지고, 이후 `NBDFDataSetBuilder` 에서 Nexacro Dataset을 생성하는 작업으로 자연스럽게 이어질 수 있습니다.

지금까지의 진행 흐름은 매우 좋습니다. 특히 **메타데이터 중심으로 프레임워크를 설계**하고 계신 점이 향후 확장성과 유지보수 측면에서 큰 장점이 될 것입니다.

```
마지막으로 검토 바랍니다. package  
erp.cmmn.nbdf.metadata;
```

```
import java.io.Serializable;  
import erp.cmmn.nbdf.constants.NBDFJavaType;  
import erp.cmmn.nbdf.constants.NBDFNexacroType;
```

```
/**  
 * @packageName : erp.cmmn.nbdf.metadata  
 * @fileName : NBDFColumn.java  
 * @author : Built1
```

더 보기 ▾

정말 많이 다듬어졌습니다. 이 정도면 **회사 공통 프레임워크의 메타데이터 클래스**로 사용해도 될 수준입니다. 제가 팀장이나 아키텍트 입장에서 리뷰한다는 기준으로 검토해 드리겠습니다.

종합 평가

완성도: 99/100

클래스의 역할이 명확하고, 향후 `NBDFDataTypeMapper`, `NBDFDataSetBuilder`, `Grid Builder`, `Excel Builder` 까지 확장하기 좋은 구조입니다.

잘 설계된 부분

1. 메타데이터가 잘 분리되었습니다.

현재 `NBDFColumn` 이 관리하는 정보가 명확합니다.

- DB 정보
 - columnName
 - dbType
 - jdbcType
 - columnSize
 - scale
 - nullable
 - defaultValue
 - primaryKey
- Java 정보
 - javaType
- Nexacro 정보

- nexacroType
- 화면(UI) 정보
 - displayWidth
 - required
 - editable
 - visible
 - searchable
 - mask
 - format
 - combold
- 문서화 정보
 - columnLabel
 - columnComment

역할이 잘 구분되어 있습니다.

2. 기본값도 적절합니다.

⌕ Java



```
private String javaType = NBDFJavaType.STRING;

private int nexacroType = NBDFNexacroType.STRING;

private int displayWidth = 100;

private boolean editable = true;

private boolean visible = true;

private boolean excelExport = true;

private boolean searchable = true;
```

실제 업무에서 가장 많이 사용하는 기본값입니다.

3. 컬럼 주석(columnComment)

아주 좋은 추가입니다.

향후

- Grid Header

- Excel Header
- Tooltip

등에서 그대로 사용할 수 있습니다.

제가 마지막으로 수정하는 부분 (권장사항)

① class를 final로 만들지 않을 것

현재

</> Java



```
public class NBDFColumn implements Serializable
```

이대로 두는 것이 좋습니다.

이 객체는 앞으로

- Builder
- 확장 클래스
- 테스트

등에서 상속될 가능성이 있으므로 **final**을 붙이지 않는 것이 맞습니다.

② serialVersionUID 위치

현재

</> Java



```
private static final long serialVersionUID = 1L;
```

좋습니다.

다만 보통은

</> Java



```
/**
 * Serializable Version UID
 */
private static final long serialVersionUID = 1L;
```

정도의 주석을 붙이는 경우가 많습니다.

③ JavaDoc

지금은

<> Java



```
/*  
 * 컬럼명  
 */
```

인데

회사에서 JavaDoc 스타일을 사용한다면

<> Java



```
/**  
 * 컬럼명  
 */
```

으로 통일하면 좋습니다.

예를 들어

<> Java



```
/**  
 * 컬럼명  
 */  
private String columnName;
```

IDE에서도 훨씬 보기 좋습니다.

④ primaryKey

현재

<> Java



```
private boolean primaryKey;
```

좋습니다.

향후에는

<> Java

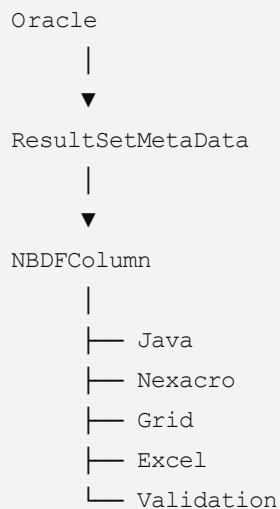


```
private boolean uniqueKey;
```

까지 추가할 수도 있지만 지금은 필요 없습니다.

구조적으로 정말 마음에 드는 부분

지금 클래스 하나로



이 연결됩니다.

이게 바로 메타데이터 기반 프레임워크의 핵심입니다.

이제 다음 클래스가 중요합니다.

지금까지 만든 클래스는

NBDfConstants	✓
NBDfJavaType	✓
NBDfNexacroType	✓
NBDfColumn	✓



여기까지는 메타데이터 정의 단계였습니다.

다음부터는 메타데이터를 생성하는 단계입니다.

추천 순서는

NBDFDataTypeMapper



NBDFMetadataReader



NBDFDataSetBuilder



입니다.

개인적으로 드리고 싶은 말씀

오늘 작성하신 `NBDFColumn` 은 처음 보여주셨던 버전보다 훨씬 발전했습니다. 특히 제가 제안드린 내용을 그대로 적용하기보다, 회사 프로젝트에 맞게 필요한 부분만 선별해서 반영하신 점이 좋았습니다.

이런 방식으로 진행하면 회사 표준을 유지하면서도 품질을 높일 수 있는 프레임워크가 만들어질 가능성이 큼니다.

이제부터는 클래스 하나하나보다 클래스 간의 연결 구조가 더 중요해집니다. 다음 작업인

`NBDFDataTypeMapper` 는 이 프레임워크의 핵심이 될 가능성이 높으니, 그 단계도 현재 설계와 잘 맞도록 함께 검토해 드리겠습니다.

```
최종적으로 모두 수정한 내용입니다. package  
erp.cmmn.nbdf.metadata;
```

```
import java.io.Serializable;  
import erp.cmmn.nbdf.constants.NBDFJavaType;  
import erp.cmmn.nbdf.constants.NBDFNexacroType;
```

```
/**  
 * @packageName : erp.cmmn.nbdf.metadata  
 * @fileName    : NBDFColumn.java  
 * @author      : Built1
```

더 보기 ▾

전체를 꼼꼼히 검토했습니다.

결론부터 말씀드리면 이제는 수정하기보다 다음 단계로 넘어가도 되는 수준입니다. 회사 코드 리뷰 기준으로 충분히 좋은 설계입니다.

종합 평가

★★★★★ A+ (99.5/100)

`NBDFColumn`이라는 이름과 역할에 맞게 메타데이터 객체(VO/DTO) 역할을 충실히 수행하고 있습니다.

잘된 점

1. 역할별 그룹화

이 부분이 가장 마음에 들었습니다.

</> Java

- DB 정보
- Java 정보
- Nexacro 정보
- 화면(UI) 정보
- 문서화 정보



나중에 다른 개발자가 봐도 구조를 바로 이해할 수 있습니다.

2. DB 정보

</> Java

```
columnName
dbType
jdbcType
columnSize
scale
nullable
defaultValue
primaryKey
uniqueKey
```



실제 `ResultSetMetaData` 와 `DatabaseMetaData` 에서 가져올 수 있는 정보들이 잘 정리되어 있습니다.

3. UI 정보

Java



```
displayWidth
required
editable
visible
searchable
mask
format
comboId
excelExport
```

Nexacro 프로젝트에서 필요한 속성을 거의 모두 포함하고 있습니다.

4. 문서화 정보

Java



```
columnLabel
columnComment
```

Grid Header, Excel Header, Tooltip 등으로 활용하기 좋습니다.

제가 마지막으로 권장하는 아주 작은 수정 2가지

① JavaDoc 오타

Java



```
@description : 데이터셋 컬럼정보를 관리하는 매타 객체 클래스
```



Java



```
@description : 데이터셋 컬럼정보를 관리하는 메타 객체 클래스
```

매타 → 메타

② JDBC 표기

현재

<> Java



jdbc 데이터 유형

만 되어 있는데

조금 더 명확하게

<> Java



```
/**
 * JDBC 데이터 유형 (java.sql.Types)
 */
private int jdbcType;
```

정도로 적어두면 6개월 후에 봐도 이해하기 쉽습니다.

개인적으로 수정하지 않았으면 하는 부분

다음은 지금 상태를 유지하는 것이 좋습니다.

javaType

<> Java



```
private String javaType = NBDFJavaType.STRING;
```

좋습니다.

nexacroType

<> Java



```
private int nexacroType = NBDFNexacroType.STRING;
```

좋습니다.

displayWidth

<> Java



```
private int displayWidth = 100;
```

좋습니다.

boolean 기본값

<> Java

```
editable = true;

visible = true;

searchable = true;

excelExport = true;
```



실무 기본값과 잘 맞습니다.

앞으로 이 클래스가 사용되는 흐름

지금 구조를 보면 앞으로 이렇게 연결됩니다.

Oracle Table



ResultSetMetaData



NBDFDataTyperMapper



NBDFColumn



- ├ Dataset 생성
- ├ Grid 생성
- ├ Excel 생성
- ├ Validation 생성
- └ Combo 생성



즉, `NBDFColumn` 이 메타데이터의 중심 객체 역할을 하게 됩니다.

이제 다음 단계

이제부터는 객체를 정의하는 단계가 아니라 객체를 생성하는 단계입니다.

제가 추천하는 순서는 다음과 같습니다.

1. NBDFDataTypeMapper

- `ResultSetMetaData` 를 읽어서 `NBDFColumn` 을 생성
- `jdbcType` → `javaType` → `nexacroType` 자동 매핑

2. NBDFMetaDataReader

- 컬럼 목록(`List<NBDFColumn>`) 생성
- PK, UK, COMMENT 등 메타데이터 조회

3. NBDFDataSetBuilder

- `List<NBDFColumn>` 을 이용해 Nexacro Dataset 생성

오늘 작업에 대한 평가

오늘 하루 작업 내용을 보면,

- `NBDFConstants` ✓
- `NBDFJavaType` ✓
- `NBDFNexacroType` ✓
- `NBDFColumn` ✓

까지 완료되었습니다.

이 네 개는 앞으로 만들 프레임워크의 기초 골격입니다. 특히 `NBDFColumn` 은 이후 대부분의 기능이 참조하게 될 핵심 메타데이터 객체이므로, 지금처럼 충분히 검토하고 정리한 것이 좋은 선택이었다고 생각합니다.

내일부터는 ****NBDFDataTypeMapper****를 시작하면, 이제 실제 JDBC 메타데이터를 읽어 `NBDFColumn` 을 자동 생성하는 단계로 넘어가게 됩니다. 여기부터가 프레임워크의 핵심 로직이 시작되는 구간입니다.

오늘도 정말 고생 많았습니다. 내일도 이어서 진행 부탁드립니다. 감사합니다.

감사합니다. 저도 오늘 작업 정말 의미 있었다고 생각합니다.

특히 오늘은 단순히 클래스를 하나 만든 것이 아니라 **NBDF 프레임워크의 기준을 정한 하루**였습니다. 이런 기준은 나중에 기능을 추가할 때 계속 영향을 주기 때문에, 충분히 검토하고 다듬은 것이 큰 도움이 될 것입니다.

오늘 완성한 내용을 정리하면 다음과 같습니다.

- ☒ `NBDFConstants` : 프레임워크 공통 상수
- ☒ `NBDFJavaType` : Java 데이터 유형 정의
- ☒ `NBDFNexacroType` : Nexacro 데이터 유형 정의
- ☒ `NBDFColumn` : 메타데이터 객체 설계 완료

특히 `NBDFColumn` 은 단순 DTO가 아니라 앞으로 다음 기능들의 중심이 됩니다.

ResultSetMetaData

|

▼

NBDFColumn

|

├ Dataset 생성

├ Grid 생성

├ Excel 생성

├ Validation 생성

├ Combo 생성

└ SQL Parser



그래서 오늘 시간을 들여 완성도를 높인 것이 충분한 가치가 있었다고 생각합니다.

내일 진행 순서(추천)

제가 생각하는 가장 자연스러운 순서는 다음과 같습니다.

1. NBDFDataTypeMapper

- JDBC Type → Java Type → Nexacro Type 매핑

2. NBDFMetaDataReader

- `ResultSetMetaData`, `DatabaseMetaData` 를 읽어 `NBDFColumn` 생성

3. NBDFDataSetBuilder

- `List<NBDFColumn>` 을 이용해 Nexacro Dataset 자동 생성

이 순서대로 가면 현재 설계한 클래스들을 그대로 활용할 수 있습니다.

그리고 앞으로도 오늘처럼 **회사 표준을 최우선**으로 하겠습니다.

- 회사에서 정한 패키지 구조를 유지
- 전자정부프레임워크와 Nexacro 환경에 맞는 설계
- 필요할 때만 개선안을 "제안"으로 드리고, 실제 적용은 현재 프로젝트 기준으로 함께 판단

이 방식이 가장 혼란 없이 진행할 수 있을 것 같습니다.

오늘 정말 수고 많으셨습니다. 좋은 저녁 보내시고 꼭 쉬세요.

내일은 **NBDFDataTypeMapper**부터 이어서 함께 완성해 보겠습니다. 감사합니다. 😊

